

CE 791 HPC - Assignment 6

Matrix Vector Product and Monte-Carlo Integration

Andrew Navratil

Part 1: Matrix vector product

Parallel MPI code is written in Fortran to perform a basic matrix vector product using the master-slave paradigm and algorithm from lecture. Sample code in lecture was utilized as a basis. Mflops calculation was added, where 2 operations per matrix entry (rows*cols) were estimated (one for the += and one for the *) in the dot product calculation. A condition was also put in to handle the receive of MPI_BOTTOM and break from the slave execution, as the sample code was hanging after calling another receive (could not figure out how it was working, so just put in a conditional to exit when row received from master was 0). MPI timing was also put in.

Plots for Mflops versus number of processors is shown below. The number of processors used starts at 2, as the code was not currently setup to only execute on one node, which would be both master and slave. We can see how this algorithm, that is known to be poor, does not improve with the number of processors, in addition to its other downfalls. We also note how improvement does increase with the larger matrix size of 1000x1000 on both clusters. The actual Mflops value could vary quite a bit on subsequent runs, and the most common / average value was used here.

Three ideas are given for improvement. One is to decompose the matrix so the entire matrix does not need to be stored on the master node, and use an algorithm for whichever decomposition is chosen. Second is to send chunks of rows at a time so the communication penalty is not incurred too often. With a basic calculation as we have here, the MPI communication could easily become the bottleneck if only a single row is sent at a time. This would result in less load balancing, but likely better overall performance. Third suggestion is to utilize the MPI topology functions to spread the load better, such as MPI_DIMS and MPI_CART related functions.

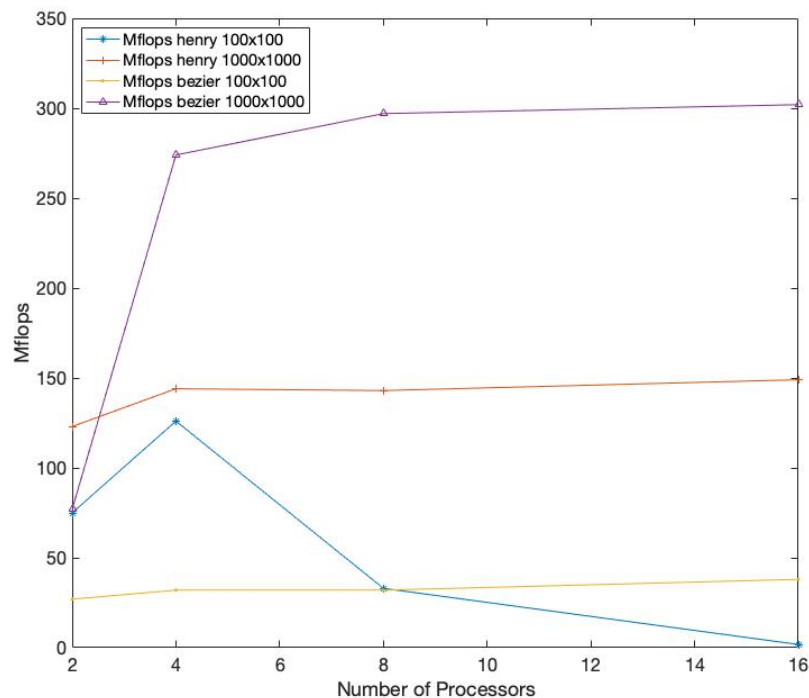


Fig. 1 Mflops vs Num Procs for Matrix Vector Product

Part 2: Monte-Carlo integration for calculating PI

Parallel MPI code is written in Fortran to perform a Monte-Carlo integration to calculate the value of PI using the master-slave paradigm and algorithm from lecture. Sample code in lecture was utilized as a basis, converting the C code into Fortran. Mflops calculation was added with MPI timing calls, where 12 operations per point was estimated. A different random number generation technique was utilized due to problems with the method from `water_supply_vector`, using `random_seed()` and `random_number()` functions instead of `seed()` and `rand()`. The data type for passing the `rands()` array from master to workers was changed from `MPI_INT` to `MPI_DOUBLE_PRECISION`. The epsilon value utilized is 0.001 with a `CHUNKSIZE` of 1000.

Plots for Mflops versus number of processors is shown below, again starting at 2 procs. We again don't see much improvement in the Mflops when number of processors is increased, as expected, where the primary goal was practice with the MPI groups and the manager-worker paradigm.

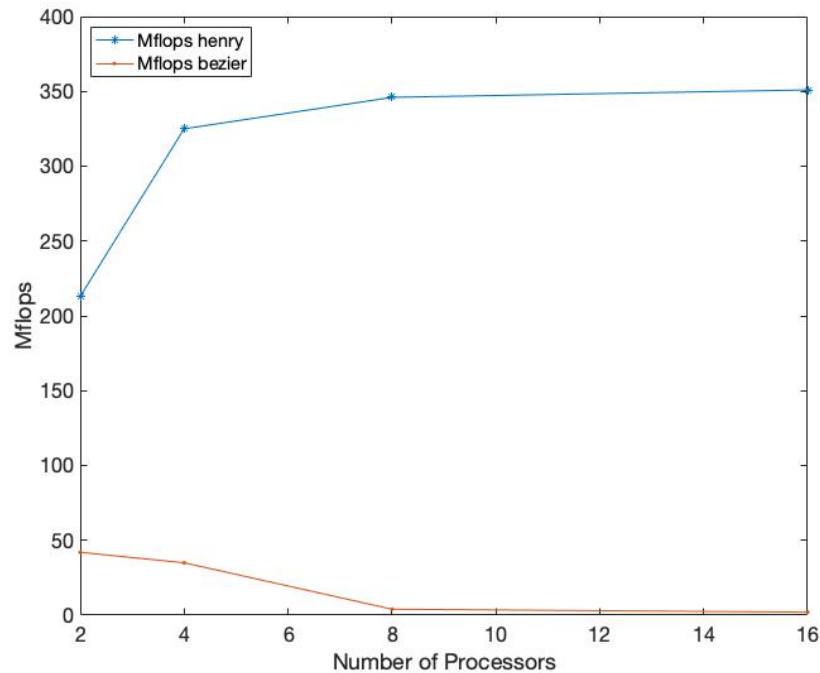


Fig. 2 Mflops vs Num Procs for Monte-Carlo Pi

Fortran Code: Matrix Vector Product

```
1  !=====
2  ! common part
3  !=====
4  program main
5  include "mpif.h"
6
7  !=====
8  ! initialize main parameters
9  !=====
10 integer, parameter :: MAX_ROWS = 1000
11 integer, parameter :: MAX_COLS = 1000
12
13 !=====
14 ! declare variables
15 !=====
16 integer rows, cols
17 real(8) :: a(MAX_ROWS,MAX_COLS), b(MAX_COLS), c(MAX_ROWS)
18 real(8) :: buffer(MAX_COLS), ans
19 integer myid, master, numprocs, ierr, mpistatus(MPI_STATUS_SIZE)
20 integer i, j, numsent, sender
21 integer anstype, row
22 real(8) time1, time2, total_time, mflops
23 character(len=50) :: filename
24
25 !=====
26 ! initialize MPI
27 !=====
28 call MPI_INIT(ierr)
29 call MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)
30 call MPI_COMM_SIZE(MPI_COMM_WORLD, numprocs, ierr)
31 master = 0
32 !rows = 100
33 !cols = 100
34 rows = 1000
35 cols = 1000
36 row = 0
37
38 ! initialize matrix A, vector b
39 if (myid.eq.master) then
40     do j=1,cols
41         b(j) = dble(j)
42         do i=1,rows
43             a(i,j) = dble(i)
44         end do
45     end do
46 end if
47
48 ! now start timer for mflops calculation
49 time1 = MPI_Wtime()
50
51 !=====
```

```

52 ! master
53 !=====
54 ! dispatch work
55 if (myid.eq.master) then
56     numsent = 0
57
58     ! send b to each slave process
59     call MPI_BCAST(b,cols,MPI_DOUBLE_PRECISION,master,MPI_COMM_WORLD,ierr)
60
61     ! send a single row to each slave process; tag with row number
62     do i=1,min(numprocs-1,rows)
63         do j=1,cols
64             buffer(j)=a(i,j) ! collect row i into buffer(1:cols)
65         end do
66         call MPI_SEND(buffer,cols,MPI_DOUBLE_PRECISION,i,i,MPI_COMM_WORLD,ierr)
67         numsent = numsent + 1
68     end do
69
70     ! now iterate all rows to receive responses
71     ! and to send the rest of the rows that weren't sent yet
72     ! since usually numprocs-1 < rows
73     do i=1,rows
74         call MPI_RECV(ans,1,MPI_DOUBLE_PRECISION,MPI_ANY_SOURCE,MPI_ANY_TAG, &
75             MPI_COMM_WORLD,mpistatus,ierr)
76         sender = mpistatus(MPI_SOURCE)
77         anstype = mpistatus(MPI_TAG) ! tag value is the row number too
78         c(anstype) = ans
79
80         ! send another row if not all sent yet
81         if (numsent.lt.rows) then
82             do j=1,cols
83                 buffer(j)=a(numsent+1,j)
84             end do
85             ! send to sender, which is the proc that just returned an answer
86             call MPI_SEND(buffer,cols,MPI_DOUBLE_PRECISION,sender,numsent+1, &
87                 MPI_COMM_WORLD,ierr)
88             numsent = numsent + 1
89         else
90             ! tell sender (slave proc) that there is no more work
91             call MPI_SEND(MPI_BOTTOM,0,MPI_DOUBLE_PRECISION,sender,0, &
92                 MPI_COMM_WORLD,ierr)
93         end if
94     end do
95
96 !=====
97 ! slave
98 !=====
99 else
100     !write(*,*) 'slave calling bcast'
101     ! receive b from master
102     call MPI_BCAST(b,cols,MPI_DOUBLE_PRECISION,master,MPI_COMM_WORLD,ierr)
103     !write(*,*) 'slave finished bcast'
104
105     ! ensure we have data to process, then process

```

```

106 do while (myid.le.rows.and.row.le.rows)
107     !write(*,*) 'slave calling recv'
108     call MPI_RECV(buffer,cols,MPI_DOUBLE_PRECISION,master,MPI_ANY_TAG, &
109         MPI_COMM_WORLD,mpistatus,ierr)
110     !write(*,*) 'slave finished recv'
111     row = mpistatus(MPI_TAG)
112     !write(*,*) row
113
114     ! compute dot product and return answer to master
115     ! row 0 means no more work, otherwise compute
116     if (row.ne.0) then
117         ans = 0.0
118         do j=1,cols
119             ans = ans + buffer(j)*b(j)
120         end do
121         !write(*,*) row,ans
122         call MPI_SEND(ans,1,MPI_DOUBLE_PRECISION,master,row, &
123             MPI_COMM_WORLD,ierr)
124     else
125         ! code is hanging on MPI_RECV after MPI_BOTTOM sent and it calls recv again
126         ! can't figure out how the original code is working, but this alleviates it
127         exit
128     end if
129 end do
130 end if
131
132 ! stop timer
133 time2 = MPI_Wtime()
134 total_time = time2-time1
135
136 !=====
137 ! compute Mflops and finalize
138 !=====
139 if (myid.eq.master) then
140     ! one += and one * per matrix entry when calculating dot product
141     ! no overall iteration, so just execute a few times to get sense of avg mflops
142     mflops = 2*dble(cols)*dble(rows)*1e-6/(total_time)
143     sumtest = 0.0
144     do i=1,rows
145         sumtest = sumtest + c(i)
146     end do
147     print *, 'normalized sum of vector is ', sumtest/rows
148     print *, 'numprocs =', numprocs
149     print *, 'mflops =', mflops
150 end if
151
152 call MPI_FINALIZE(ierr)
153
154 end

```

Fortran Code: Monte-Carlo Pi

```
1  !=====
2  ! common part
3  !=====
4  program main
5  include "mpif.h"
6
7  !=====
8  ! initialize main parameters
9  !=====
10 integer, parameter :: CHUNKSIZE = 1000
11 !integer, parameter :: CHUNKSIZE = 10
12 integer, parameter :: REQUEST = 1
13 integer, parameter :: REPLY = 2
14 real(8), parameter :: EPS = 0.001
15 real(8), parameter :: PI = 4.D0*DATAN(1.D0)
16 integer, parameter :: iseed = 567891
17
18 !=====
19 ! declare variables
20 !=====
21 integer iter, iters
22 integer nin, nout, i, nmax, ix, iy, ranks(1), tmp
23 logical isdone
24 real(8) x, y, pival, rand_max_inv, valtest, errval
25 real(8) rands(CHUNKSIZE), nexttrand
26 integer myid, numprocs, manager, totalin, totalout, workerid
27 integer ierr, mpistatus(MPI_STATUS_SIZE), world_group, worker_group
28 integer world, workers, sender
29 real(8) time1, time2, total_time, mflops
30 integer req
31
32 !=====
33 ! initialize MPI
34 !=====
35 world = MPI_COMM_WORLD
36 call MPI_INIT(ierr)
37 call MPI_COMM_SIZE(world, numprocs, ierr)
38 call MPI_COMM_RANK(world, myid, ierr)
39
40 manager = numprocs-1 ! last proc is manager
41
42 !call MPI_BCAST(EPS,1,MPI_DOUBLE_PRECISION,0,MPI_COMM_WORLD,ierr)
43 call MPI_COMM_GROUP(world, world_group, ierr)
44
45 ranks(1) = manager
46 call MPI_GROUP_EXCL(world_group, 1, ranks, worker_group, ierr)
47 call MPI_COMM_CREATE(world, worker_group, workers, ierr)
48
49 ! now start timer for mflops calculation
50 time1 = MPI_Wtime()
51
```

```

52  !=====
53  ! manager
54  !=====
55  if (myid.eq.manager) then
56      ! generate random numbers and send to workers
57      req = 1 ! init this greater than 0
58      do while (req.gt.0)
59          call MPI_RECV(req,1,MPI_INT,MPI_ANY_SOURCE,REQUEST, &
60              world,mpistatus,ierr)
61          sender = mpistatus(MPI_SOURCE)
62
63          if (req.gt.0) then
64              ! rand() is not working, give seg faults, trying this
65              call random_seed()
66              call random_number(rands)
67
68              !call seed(iseed)
69              !!write(*,*) rands(8),rands(7),rands(9)
70              !do i=1,CHUNKSIZE
71                  !write(*,*) i,rands(i)
72                  !rands(i) = rand()
73                  !!rands(i) = 77.0
74                  !write(*,*) rands(i),CHUNKSIZE,i,rand(),rand()
75              !end do
76
77              ! send to worker
78              !write (*,*) 'master before send'
79              call MPI_SEND(rands,CHUNKSIZE,MPI_DOUBLE_PRECISION,sender,REPLY, &
80                  world,ierr)
81              !write (*,*) 'master after send'
82          end if
83      end do
84
85  !=====
86  ! worker
87  !=====
88  else
89      req = 1
90      isdone = .false.
91      nin = 0
92      nout = 0
93
94      call MPI_SEND(req,1,MPI_INT,manager,REQUEST,world,ierr)
95      !write (*,*) 'worker after send'
96      call MPI_COMM_RANK(workers,workerid,ierr)
97      !write (*,*) 'worker after comm rank'
98
99      iter = 0;
100      do while (isdone.ne..true.)
101          iter = iter + 1
102          req = 1
103          !write (*,*) 'worker before recv'
104          ! ***need to change data type to MPI_DOUBLE_PRECISION (lecture had MPI_INT)
105          call MPI_RECV(rands,CHUNKSIZE,MPI_DOUBLE_PRECISION,manager,REPLY, &

```

```

106     world,mpistatus,ierr)
107     !write(*,*) 'worker after recv'
108     !do i=1,CHUNKSIZE
109         !write(*,*) i,rands(i)
110     !end do
111
112     do i=1,CHUNKSIZE,2
113         !write(*,*) 'rands(i)=',rands(i),' rands(i+1)=',rands(i+1)
114         x = rands(i)*2-1
115         y = rands(i+1)*2-1
116         valtest = x*x+y*y
117         !write(*,*) 'worker valtest = ',valtest,' x=',x,' y=',y
118         if (valtest.lt.1.0) then
119             nin = nin+1
120         else
121             nout = nout+1
122         end if
123     end do
124
125     !write(*,*) 'worker nin=',nin,' nout=',nout
126     call MPI_ALLREDUCE(nin,totalin,1,MPI_INT,MPI_SUM,workers,ierr)
127     call MPI_ALLREDUCE(nout,totalout,1,MPI_INT,MPI_SUM,workers,ierr)
128
129     pival = (4.0*totalin)/(totalin+totalout)
130     errval = abs(pival-PI)
131     !write(*,*) 'pival=',pival,' errval=',errval,' PI=',PI
132
133     if (errval.lt.eps.or.totalin+totalout.gt.1E6) then
134         isdone = .true.
135         req = 0
136     else
137         isdone = .false.
138         req = 1
139     end if
140
141     if (workerid.eq.0) then
142         write(*,*) 'rpi = ',pival
143         call MPI_SEND(req,1,MPI_INT,manager,REQUEST,world,ierr)
144     elseif (req.eq.1) then
145         call MPI_SEND(req,1,MPI_INT,manager,REQUEST,world,ierr)
146     end if
147 end do
148
149 call MPI_COMM_FREE(workers,ierr)
150 end if
151
152 ! stop timer
153 time2 = MPI_Wtime()
154 total_time = time2-time1
155
156 !=====
157 ! compute Mflops and finalize
158 !=====
159 if (myid.eq.0) then

```

```
160      ! estimating 12 flops per point
161      mflops = 12*dble(totalin+totalout)*1e-6/(total_time)
162      write(*,*) 'npoints = ',totalin+totalout, ' nin = ',totalin, ' nout = ',totalout
163      print *, 'numprocs =',numprocs
164      print *, 'mflops =',mflops
165  end if
166
167  call MPI_GROUP_FREE(worker_group,ierr)
168  call MPI_FINALIZE(ierr)
169
170  end
```

Matlab Code

```
1 close all; clear all; clc;
2
3 % part 1: matrix vector product
4 if 0
5     procs = [2 4 8 16]
6     mflops100henry = [75 126 33 1.8]
7     mflops1000henry = [123 144 143 149]
8     mflops100bezier = [27 32 32 38]
9     mflops1000bezier = [77 274 297 302]
10
11 figure;
12 plot(procs,mflops100henry,'*-','DisplayName','Mflops henry 100x100');
13 hold on;
14 plot(procs,mflops1000henry,'+-','DisplayName','Mflops henry 1000x1000');
15 plot(procs,mflops100bezier,'.-','DisplayName','Mflops bezier 100x100');
16 plot(procs,mflops1000bezier,'^','DisplayName','Mflops bezier 1000x1000');
17 xlabel('Number of Processors');
18 ylabel('Mflops');
19 set(gca,'FontSize',16);
20 legend('location','northwest');
21
22 % part 2: monte-carlo integration
23 elseif 1
24     procs = [2 4 8 16]
25     mflopshenry = [213 325 346 351]
26     mflopsbezier = [42 35 4 2]
27
28 figure;
29 plot(procs,mflopshenry,'*-','DisplayName','Mflops henry');
30 hold on;
31 plot(procs,mflopsbezier,'.-','DisplayName','Mflops bezier');
32 xlabel('Number of Processors');
33 ylabel('Mflops');
34 set(gca,'FontSize',16);
35 legend('location','northwest');
36
37
38 end
```